

引用格式: 张学军, 梁书滨, 白万荣, 等. 基于异构图表征的源代码漏洞检测方法 [J]. 浙江大学学报: 工学版, 2025, 59(8): 1644–1652.

Citation: ZHANG Xuejun, LIANG Shubin, BAI Wanrong, et al. Source code vulnerability detection method based on heterogeneous graph representation [J]. Journal of Zhejiang University: Engineering Science, 2025, 59(8): 1644–1652.

基于异构图表征的源代码漏洞检测方法

张学军¹, 梁书滨¹, 白万荣², 张奉鹤¹, 黄海燕¹, 郭梅凤¹, 陈卓¹

(1. 兰州交通大学 电子与信息工程学院, 甘肃 兰州 730070;

2. 国网甘肃省电力公司 电力科学研究院, 甘肃 兰州 730070)

摘 要: 针对现有的源代码漏洞检测模型对异构特征和底层信息提取不足导致的检测准确率不高的问题, 提出基于异构图表征的源代码漏洞检测方法. 从中间代码表示 (IR) 中提取 8 种指令级特征作为程序依赖图的节点嵌入, 解决底层信息提取不足的问题. 在节点层和依赖层分别构建基于注意力机制的聚合模块来提取图表征数据中的异构性特征, 通过调整注意力系数捕获关键节点信息. 对图数据的聚合结果进行分类, 预测是否存在漏洞. 在合成数据集和 2 个真实项目数据集上的实验表明, 相比于现有方法, 本文方法具有更强的异构特征提取能力和更高的漏洞检测综合性能.

关键词: 漏洞检测; 图表征; 注意力机制; 异构特征; 中间代码表示

中图分类号: TP 391

文献标志码: A

文章编号: 1008-973X(2025)08-1644-09

DOI: 10.3785/j.issn.1008-973X.2025.08.011

Source code vulnerability detection method based on heterogeneous graph representation

ZHANG Xuejun¹, LIANG Shubin¹, BAI Wanrong², ZHANG Fenghe¹, HUANG Haiyan¹,

GUO Meifeng¹, CHEN Zhuo¹

(1. College of Electronic and Information Engineering, Lanzhou Jiaotong University, Lanzhou 730070, China; 2. Electric Power Research Institute, State Grid Gansu Electric Power Company, Lanzhou 730070, China)

Abstract: A source code vulnerability detection method based on heterogeneous graph representation was proposed aiming at the problem of low detection accuracy caused by the insufficient extraction of heterogeneous features and low-level information in the existing source code vulnerability detection models. Eight instruction-level features were extracted from the intermediate code representation (IR) to serve as node embeddings for the program dependence graph, which addressed the issue of missing low-level information. Attention aggregation mechanisms were constructed at the node level and dependency level respectively to extract heterogeneous features, and information of key nodes was extracted by adjusting attention coefficients. The aggregated results of the graph data were classified to predict the presence of vulnerability. The experiments on synthetic data sets and two real project data sets show that the proposed method has stronger capabilities in extracting heterogeneous features and higher comprehensive performance in vulnerability detection compared with existing methods.

Keywords: vulnerability detection; graph representation; attention mechanism; heterogeneous feature; intermediate code representation

随着市场对软件安全需求的提升, 针对软件漏洞的检测成为该领域的研究热点^[1]. 近年来, 国内外学者提出使用静态测试和动态分析的方法对源

代码漏洞进行检测与分析. 动态分析在面对大型复杂软件系统的漏洞时, 存在检测率较低的问题. 与静态测试相比, 静态测试能够在不实际运行程序

收稿日期: 2024-08-21.

网址: www.zjujournals.com/eng/article/2025/1008-973X/202508011.shtml

基金项目: 国家自然科学基金资助项目 (61762058, 62461032); 甘肃省教育厅产业支撑项目 (2022CYZC-38); 甘肃省重点研发计划资助项目 (25YEFA089); 国家电网科技资助项目 (W32KJ2722010, 522722220013).

作者简介: 张学军 (1977—), 男, 教授, 博导, 从事网络安全、数据隐私与机器学习的研究. orcid.org/0000-0002-0350-359X.

E-mail: xuejunzhang@mail.lzjtu.cn

的情况下发现源代码中的安全漏洞,解决了动态分析无法完全覆盖所有代码的问题,得到了研究者的高度重视。静态测试主要包括模式匹配^[2]、规则检查^[3]和基于学习的方法^[4]。模式匹配和规则检查严重依赖人工,对未定义的漏洞模式缺乏识别能力。利用基于学习的方法,能够有效避免因人工定义导致的漏洞模式不全的问题,该方法成为静态测试的主要手段之一。

Li等^[5]利用双向长短时记忆网络检测漏洞,但其严重依赖上下文,捕获长距离依赖关系的能力有限。为了提取更丰富的语法和语义信息, Sy-SeVR扩充了4种漏洞语法规则^[6]。上述方法只在源代码层面提取漏洞特征,缺乏对底层逻辑的有效提取。Li等^[7]从中间代码表示(IR)中提取特征,构建样本切片和代码语句之间的关联,实现了细粒度检测。将IR视为纯文本会影响模型对程序底层逻辑的学习,且过短的切片不利于漏洞信息的远距离传播。Wu等^[8-9]分别采用卷积神经网络和图神经网络(graph neural network, GNN)提取特征,取得了更高的准确率,但传统GNN对异质图的泛化能力不足。Wang等^[10-11]采用改进的GNN,通过拼接方式融合异构依赖关系。拼接会使得依赖层异质性缺失,降低漏洞模式的自适应能力。

针对上述问题,本文提出基于异构图表征的源代码漏洞检测方法(vulnerability detection based on heterogeneous graph, VulHetG)。生成源代码对应的IR,提取8种指令级别的特征用于代替源代码特征空间构造,解决了源代码底层信息缺失导致的检测率低的问题。分别在PDG的节点层和依赖层构建基于图注意力网络(graph attention network, GAT)和自注意力的分层聚合机制,考虑每个异构因子的重要性。通过调整注意力系数来提高关键节点的权重,降低因传统切片引起的长距离依赖损失。在合成数据集和真实数据集上的实验结果表明, VulHetG在合成数据集上的准确率较5种基线方法分别提高了1%、1.3%、2%、17.5%和26.3%,在真实数据集上的准确率较3种基线方法分别提高了7%、38%和45.5%。

1 相关工作

基于规则的检测方法通过预定义的漏洞规则,对源代码文件进行遍历和匹配,找到潜在的漏洞威胁,但此类方法对漏洞规则具有很强的依赖性。检测工具PMD^[12]涵盖了编码规范、设计原则和安全漏洞等规则,以应对复杂的代码场景。工具SonarQube^[13]在可靠性、适用性方面进行了提升,不仅

提供了多维度的代码质量评估,而且支持多种编程语言的检测任务。基于学习的检测方法分为基于机器学习和基于深度学习的检测方法。Perl等^[14]将GitHub提交记录映射到已知漏洞,使用支持向量机来标记可能存在漏洞的样本,但该方法在实际检测中存在较高的假阳性率。相比之下,基于深度学习的方法更有利于深层次特征的提取。

基于深度学习的检测方法通常采用Token或图形的方式对源代码进行表征。 μ VulDeePecker^[15]在VulDeePecker^[5]的基础上引入代码小工具来细化漏洞特征提取,获得了多类型漏洞检测能力。相比于Token,图表征对代码结构的表达更加清晰和显式化。Feng等^[16]验证了多种GNN模型,获得了优于Token表征的检测结果,但GNN难以充分提取图表征中的异构信息。徐泽鑫等^[17]通过解耦、融合的方式获得了高质量特征,并利用距离损失放大漏洞样本的差异性降低了误报率。程靖云等^[18]以不相交图的形式拼接子图,采用多头图注意力网络(multi-head graph attention network, MHGAT)跨图层传递信息,有效提取了图数据异构特征。以上方法虽然考虑了图数据异构性,但存在特征分布稀疏问题。Wang等^[10]利用门控图神经网络(gated graph neural network, GGNN),在表示特定依赖的子图之间提取异构特征。Fan等^[11]采用圆形GGNN融合张量中的异构信息。虽然这2种方法对图数据异构特征的考虑更深入,但缺乏对异构元素的独立提取和重要性评估,导致隐秘的漏洞节点和边在聚合过程中被忽视。本文针对异构特征提取不充分的问题,提出基于异构图的源代码漏洞检测方法。

2 VulHetG 方法的设计

VulHetG方法的整体框架如图1所示,分为4个部分:表征构建、特征嵌入、节点层聚合、依赖层聚合与漏洞检测。

2.1 表征构建

如图1(a)所示,本文通过底层虚拟机(low-level virtual machine, LLVM)编译器前端Clang^[19],生成中间代码LLVM-IR^[20]和AST,利用Pass^[21]提取依赖信息以生成PDG。

为了解决底层信息提取不足的问题,从Mirsky等^[22]提供的C/C++项目中,提取8种指令级特征:节点编号、内置函数、静态赋值、操作指令、条件分支、变量、变量类型、原函数。以8种指令级特征之一的“操作指令”为例,与源代码信息进行对比,如表1所示。

“操作指令”具有确定数量的特征项,通过指令

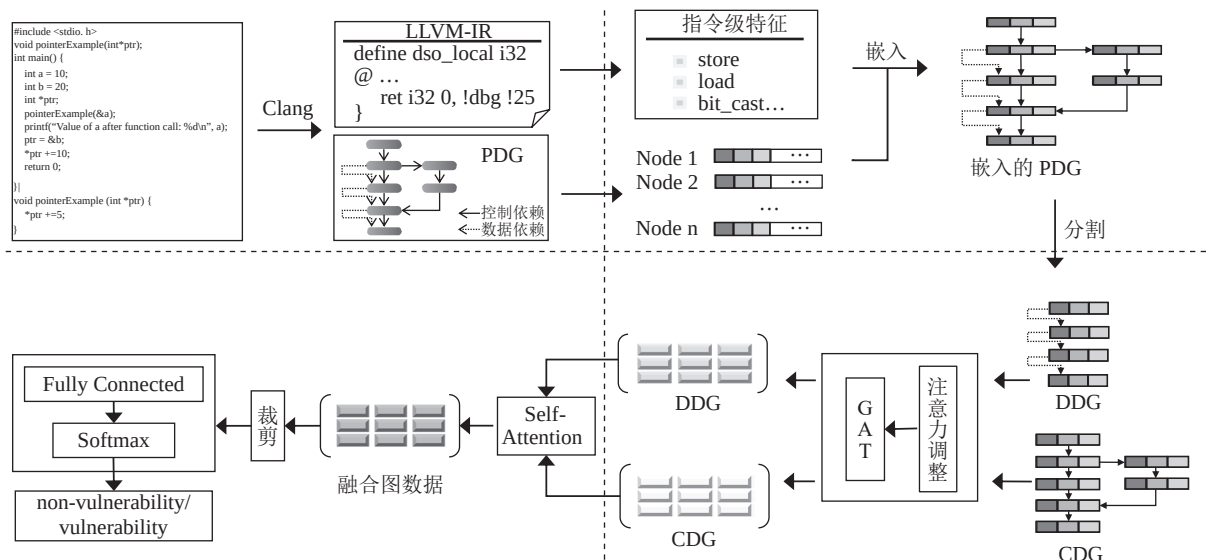


图 1 VulHetG 方法的整体架构

Fig.1 Framework of VulHetG method

表 1 “操作指令”与源代码信息的对比

Tab.1 Comparison of “operation instructions” and source code

类型	LLVM-IR	数量	源代码(举例)
算数指令	add,sub...	12	+, -, *, /, %
位指令	shl,lshr...	6	<<, >>, &, , ^
转换指令	trunc,zext...	9	char b = 97
内存指令	load,store...	3	free *ptr
比较指令	icmp,fcmp...	2	>, <, ==
分支指令	call,ret,br...	3	goto label
异常处理	landingpad...	2	std::exception
向量指令	llvm.vector...	3	vec[3] = 5
原子指令	atomicrmw...	3	fetch_add()
聚合指令	insertvalue...	4	struct S{float x}
其他指令	select,phi...	3	condition ? a : b

名称可以精确地描述程序的运行逻辑,例如 load、store 和 alloca 清晰地描述了内存的读取、存储和分配.这 8 种指令级特征是通过统计所有项目中的 LLVM-IR 指令并对其分类后得到,它们与程序运行逻辑密切相关,能够覆盖的漏洞范围超过了 4 种漏洞语法规则^[6].

2.2 特征嵌入

为了将特征作用到 PDG 的每个节点中,设计特征嵌入算法,相关定义如下.

定义 1 源代码. $P = \{p_1, p_2, p_3, \dots, p_n\}$, 其中 $p_i (1 \leq i \leq n)$ 表示源代码中的一条语句,由一组令牌序列构成,包括变量标识符、函数标识符、常量、关键字及运算符.

定义 2 漏洞语法规则. $R = \{r_k\}, 1 \leq k \leq 4$. 其中 R 表示 4 种语法规则, r_1 表示 API/库函数调用, r_2 表示指针调用, r_3 表示算数表达式, r_4 表示数

组操作.

定义 3 指令级特征. $L = \{l_m\}, 1 \leq m \leq 8$. 其中 L 表示从 LLVM-IR 中提取的 8 种指令级特征: 节点编号、操作指令、内置函数、静态赋值、条件分支、变量、类型、原函数.

定义 4 嵌入后的 PDG. 嵌入特征的图数据用 $G = \{f_T, M_{DDG}, M_{CDG}\}$ 表示, 其中 f_T 为特征张量, M_{DDG} 和 M_{CDG} 为数据依赖和控制依赖的邻接矩阵.

为了将特征作用于 PDG 的节点, VulHetG 将从 LLVM-IR 提取的 8 种指令级特征嵌入到 PDG 节点中, 具体嵌入过程如算法 1 所示.

算法 1 特征嵌入算法

输入: 源代码 P 、语法规则 R 、指令级特征 L

输出: $G = \{f_T, M_{DDG}, M_{CDG}\}$ 、集合 N_G

- 1) 根据 P 生成 PDG 和 LLVM-IR, PDG 节点集合为 N , 单个节点 $n_i \in N$.
- 2) $M_{DDG} \leftarrow$ 提取 PDG 中的数据依赖关系.
- 3) $M_{CDG} \leftarrow$ 提取 PDG 中的控制依赖关系.
- 4) $f_T \leftarrow \emptyset, N_G \leftarrow \emptyset$
- 5) for each $n_i \in N$ do:
- 6) $f_i \leftarrow \emptyset$
- 7) $IR_i \leftarrow$ LLVM-IR 中 n_i 对应的片段
- 8) for each $l_m \in L$ do:
- 9) $f_i \leftarrow f_i \oplus \text{Word2Vec}(IR_i \cap l_m)$
- 10) end for
- 11) $f_T \leftarrow$ 堆叠 (f_i, f_i)
- 12) if $n_i \in R$ then
- 13) $N_G \leftarrow N_G \cup n_i$
- 14) end if
- 15) end for

16) return $G = \{f_T, M_{DDG}, M_{CDG}\}, N_G$

步骤1)表示准备阶段,根据源代码生成对应的PDG和LLVM-IR.

步骤2)、3)表示生成阶段,提取PDG中不同的依赖关系,构建数据依赖图(data dependency graph, DDG)和控制依赖图(control dependency graph, CDG)对应的邻接矩阵.

步骤4)~11)表示嵌入阶段,先为图数据定义特征空间 f_T 和节点候选集 N_G ,然后遍历PDG节点,在每个节点对应的LLVM-IR片段 IR_i 上提取 L 中的特征,共得到8个不同的特征值.利用Word2Vec算法将每个特征值转化为一维向量,再将它们拼接成1个完整的一维向量作为节点特征 f_i 的填充.最后,将所有的节点嵌入堆叠成1个二维张量,将它作为图数据特征空间 f_T 的填充.

步骤12)~14)为关键节点提取阶段,根据漏洞语法规则 R ,在遍历过程中提取符合语法规则的节点,并纳入候选集 N_G 中.

步骤16)为输出阶段,返回嵌入后的图 G 和节点集 N_G .

2.3 节点层聚合

PDG经过特征嵌入,输出图 G 和节点集 N_m ,先将 G 切分为 G_1 和 G_2 :

$$G_1 = \{f_T, M_{DDG}\}, \quad (1)$$

$$G_2 = \{f_T, M_{CDG}\}. \quad (2)$$

在节点聚合层, G_1 和 G_2 的聚合过程相同,这里以 G_1 为例进行说明.GAT先依据 M_{DDG} 确定存在依赖关系的节点 i 和 j ,再依据 i 和 j 分别在 f_T 中计算两点之间的注意力系数 e_{ij} :

$$e_{ij} = a^T (Wh_i \oplus Wh_j). \quad (3)$$

式中: h_i 和 h_j 分别为节点 i 和 j 的特征向量, a 和 W 分别为可学习的系数向量和权重矩阵.为了提高 N_G 中节点在聚合过程中的关注度,对相关的注意力系数进行调整.

$$\tau[j] = \begin{cases} 2, & j \in N_G; \\ 1, & \text{其他}. \end{cases} \quad (4)$$

$$e_{ij} = \tau[j] \cdot e_{ij}. \quad (5)$$

式中: τ 为调整系数.在本文的设计中,当节点 j 属于 N_G (符合4种漏洞语法规则的节点集)时, $\tau=2$,即 e_{ij} 变为原来的2倍,从而在GAT网络聚合邻域信息时,提高对节点 j 的关注度;反之,则保持原来的 e_{ij} 不变.该注意力系数调整方法借鉴了可解释性的研究工作^[23],利用干预学习中的权重调整操作,对重要节点的注意力系数进行倍数扩大来提

高重要节点的聚合权重.式(6)的归一化计算保证了系数调整的合理性:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(e_{ij}))}{\sum_{v \in N(i)} \exp(\text{LeakyReLU}(e_{iv}))}. \quad (6)$$

式中: $N(i)$ 为节点 i 的所有邻居节点集合;LeakyReLU为非线性激活函数; $\exp$ 为指数函数,作用是放大注意力系数之间的差异.在得到节点的 α_{ij} 后,计算多头注意力的输出结果,得到节点 i 的更新向量表示:

$$h_i' = \parallel_{k=1}^K \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^k W^k h_j \right). \quad (7)$$

式中: σ 为激活函数, K 为注意力头个数, α_{ij}^k 和 W^k 分别为第 k 个注意力头对应的权重和线性变换矩阵, h_i' 为更新后节点 i 的表示.

经过上述过程, G_1 中的所有节点向量都得到了更新,最终输出由节点向量构成的二维张量 T_{DDG} . G_2 经过和 G_1 相同的过程,得到张量 T_{CDG} . G_1 和 G_2 的聚合过程同时进行,GAT根据 M_{CDG} 和 f_T 计算 G_2 中各点的注意力系数并进行调整.

2.4 依赖层聚合

为了考虑PDG各依赖关系的异构性,Vul-HetG基于注意力机制构建依赖聚合层.对于节点层聚合产生的2个图张量 T_{DDG} 和 T_{CDG} ,尺寸均为 (N, D) ,其中 N 为节点数, D 为特征维度.为了聚合张量 T_{DDG} 和 T_{CDG} ,计算 T_{DDG} 和 T_{CDG} 中的每个元素 t_d 和 t_c 间的注意力分数:

$$\text{score}(t_d, t_c) = \frac{t_d \cdot t_c^T}{\sqrt{D}}. \quad (8)$$

式中: t_d 和 t_c^T 的点积为注意力分数.利用注意力权重,对 t_d 进行更新:

$$t_d' = \sum_{c=1}^D \text{softmax}(\text{score}(t_d, t_c)) \cdot t_c. \quad (9)$$

将softmax归一化的权重矩阵与对应的元素 t_c 相乘得到加权向量,最终得到元素 t_d 的更新 t_d' .对于 T_{CDG} 中的元素 t_c ,以相同的方式更新:

$$t_c' = \sum_{d=1}^D \text{softmax}(\text{score}(t_c, t_d)) \cdot t_d. \quad (10)$$

据此, T_{DDG} 和 T_{CDG} 都对各自的元素进行了更新,得到新的张量表示 T_{DDG}' 和 T_{CDG}' .通过拼接的方式将二者融合,融入节点层和依赖层异构特征的张量为

$$T = T_{DDG}' \oplus T_{CDG}'. \quad (11)$$

在检测阶段,针对融合张量 $T(N, 2D)$ 构建分类

任务. 因数据样本包含的节点数不同, 导致维度 N 不同, 为此计算 T 的均值, 并按顺序裁剪.

$$S = \left\{ \frac{1}{2D} \sum_{y=1}^{2D} t_{xy} \right\}; 1 \leq x \leq N. \quad (12)$$

$$T' = T [\text{SelectTopN}(S, n)]. \quad (13)$$

式中: t_x 为 T 中的向量, t_{xy} 为 t_x 中的元素; S 为由 t_x 各元素的均值构成的集合; SelectTopN 操作用来筛选最大的前 n 个元素; n 为 200, 表示将每个图的节点数控制为 200.

2.5 漏洞检测

根据异构图表征的特性, 设计有效的漏洞检测模型, 具体结构如图 2 所示.

在节点层, 调整部分节点的注意力系数, 得到权重矩阵 W'_i 和 W''_i ($1 \leq i \leq n$), 其中 n 为网络层数. 为了减小传递误差, 采用 3 个注意力头, 利用 Dropout 正则化技术, 减小模型对特定样本的依赖. 在依赖层, 计算 CDG 和 DDG 各自的注意力权重, 将二者加权融合. 漏洞检测模块由全连接层和 Softmax 激活层构成, 融合的张量特征在该阶段被深度提取. 采用交叉熵损失函数计算损失.

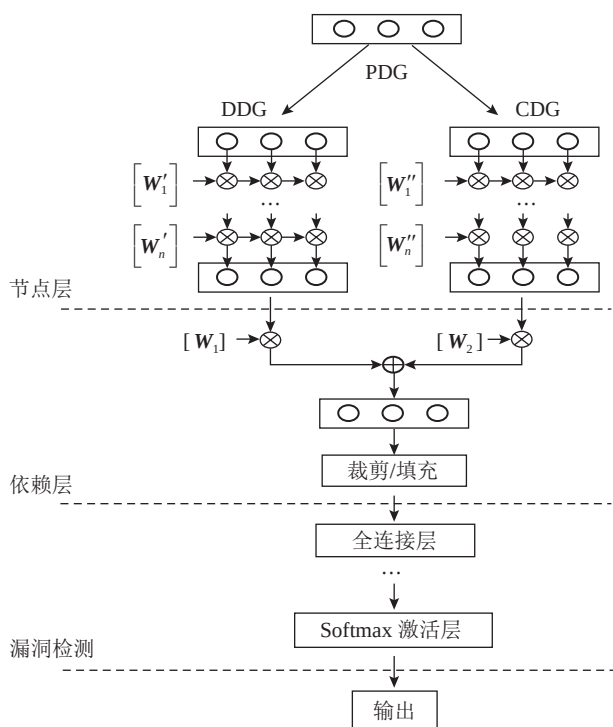


图 2 漏洞检测模型的结构

Fig.2 Structure of vulnerability detection model

3 实验分析

实验采用 2 个合成数据集和 2 个真实数据集. 合成数据集包括自建数据集和 SySeVR 数据集, 真实数据集包括 Devign 和 REVEAL^[24].

1) 自建数据集: 从合成漏洞数据库 SARD 和 NVD 中提取 28 830 条 C/C++ 数据样本, 共生成 15 710 个 PDG 样本, 包括纯净 PDG 样本 9 887 个, 10 类漏洞 PDG 样本共 5 823 个. CWE-ID 具有类似树形结构的层次, 层次越高表示漏洞场景越广泛. 在第 3 层挑选发生几率最高的 10 个 CWE-ID 进行实验.

2) SySeVR 数据集: 该数据集基于 SARD 漏洞库, 根据 4 种漏洞语法规则切片得到. 这 4 种语法规则分别是 AE (算术表达式)、AU (数组使用)、FC (API/库函数调用)、PU (指针操作). 由于语法规则的广泛适应性, 该数据集能够覆盖 93.6% 以上的漏洞场景.

3) 真实数据集: REVEAL 是从 2 个开源项目的历史漏洞中构建的数据集, 其对内存管理、输入验证相关的漏洞具有较强的覆盖能力. Devign 是从 4 个开源的 C 语言项目中构建的数据集, 结合多种代码表示, 提供了全面的漏洞语义信息.

在合成数据集上, 采用准确率 Acc、精度 P 、 F_1 和召回率 R 作为评价指标. 在真实项目数据集上, 采用误报率 FPR 和漏报率 FNR 进行评价.

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FN} + \text{FP}}, \quad (14)$$

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad (15)$$

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (16)$$

$$F_1 = 2 \times \frac{PR}{P + R}, \quad (17)$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (18)$$

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}. \quad (19)$$

式中: TP 为正样本被正确预测的数量, TN 为负样本被正确预测的数量, FP 为正样本被错误预测的数量, FN 为负样本被错误预测的数量.

3.1 针对 10 类漏洞的检测性能

为了验证 VulHetG 对各类型漏洞的检测能力, 在包含 10 类漏洞的数据集上进行实验验证, 结果如表 2 所示.

从表 2 可以看出, 涉及内存管理错误的 3 类漏洞 CWE-476、CWE-119 和 CWE-404 的平均 Acc 和 F_1 最高, 达到 96.6% 和 96.9%. 原因是 VulHetG 从中间代码维护了丰富的内存信息, 导致模型对错误的内存操作较敏感. 例如, 当发生越界访问时, 内存操作指令将错误的参数机器码传递给处理器, 返回实际分配之外的地址, 并在中间代码中以变量符的形式反映出来, 有助于模型学习漏洞特征.

表 2 对 10 类漏洞的检测

Tab.2 Detection of 10 types of vulnerabilities %

CWE编号	Acc	P	F_1	R
CWE-476	96.87	97.52	96.97	96.43
CWE-706	95.73	95.81	95.55	94.85
CWE-119	96.41	96.49	97.35	98.23
CWE-404	96.55	94.83	96.49	98.21
CWE-665	96.92	97.06	97.06	97.06
CWE-074	95.38	96.97	95.52	94.11
CWE-020	93.75	93.85	93.88	93.91
CWE-400	95.45	93.75	95.74	97.82
CWE-311	86.66	90.59	86.20	80.64
CWE-704	95.83	96.02	96.02	96.02

CWE-311 和 CWE-704 是由加密和身份验证引发的漏洞, Acc 相差约 9.2%. 这是因为 CWE-704 漏洞的发生与跨平台的类型转换有关, 而中间代码的 `llvm.module.flags` 内置函数具有定义目标平台数据布局的功能, 有利于 CWE-704 漏洞特征的提取.

从整体来看, VulHetG 对 10 类具有代表性的漏洞平均检测 Acc 和 F_1 分别达到 94.9% 和 95.1%, 这表明 VulHetG 对涉及内存、输入验证、资源管理和权限管理的常见漏洞普遍具有较高的检测能力.

3.2 与基线方法的性能比较

资源管理错误漏洞 CWE-399 和缓冲区溢出漏洞 CWE-119 在引发方式方面具有一定的代表性. 为了验证 VulHetG 特征提取方式的有效性, 通过实验对比了 5 种基线模型, 结果如表 3 所示. 表中, DT 为单个样本的平均检测时间.

从表 3 可以看出, VulHetG 在 2 类漏洞上的 Acc 达到 94% 和 96%, 高于双通道模型 mVulSniffer^[25]. 这可能是因为中间代码中, 变量在每次参与运算时都会附带完整的变量信息, 且会用地地址指针代替变量作为传参, 在多数情况下, 模型只需要跨越很短的距离就可以提取依赖关系.

VulHetG 模型的检测指标均优于 VulDeePecker 和 SySeVR, 这是因为 GAT 比 LSTM 和 BGRU 更适合提取图数据特征. 此外, 图表征形式为 VulHetG 提供了更丰富的结构和依赖信息.

FUNDED 和 VDoTR 比 3 种基于 Token 的基线方法检测用时更长, 原因是图特征提取更复杂. VulHetG 对 CWE-399 的 Acc 略低于 VDoTR, 这是因为张量表征形式的泛化能力更加稳定.

表 3 对 CWE-399 和 CWE-119 的检测性能对比

Tab.3 Comparison of detection for CWE-399 and CWE-119

模型	CWE-399				CWE-119			
	Acc/%	P/%	F_1 /%	DT/s	Acc/%	P/%	F_1 /%	DT/s
VulDeePecker	71.29	75.18	71.31	0.885	73.06	76.24	73.31	0.891
SySeVR	75.94	80.56	78.24	1.092	79.80	84.62	79.20	1.105
mVulSniffer	92.73	86.35	85.90	1.380	95.32	93.09	88.36	1.378
FUNDED	93.28	93.70	93.41	2.156	93.80	94.40	94.38	2.108
VDoTR	94.12	94.27	94.25	2.361	94.75	96.81	95.65	2.258
VulHetG	94.05	94.16	94.62	2.173	96.01	96.13	95.87	2.112

3.3 在 4 种语法规则上的性能对比

为了验证 VulHetG 在异构图数据处理中的有效性, 在包含 4 种漏洞语法规则的数据集上对比 5 种基线方法, 实验结果如表 4 所示.

从表 4 可以看出, 与 VulDeePecker、SySeVR 和

mVulSniffer 方法相比, VulHetG 的平均检测准确率分别提升了 29.6%、19.3% 和 2.6%, 比 FUNDED 和 VDoTR 方法的平均检测精度分别提升了 1.6% 和 0.9%. 这是因为 FUNDED 和 VDoTR 基于 GGNN 网络构建模型, 相比于 GAT 加自注意力机制的模型

表 4 对 4 种语法规则漏洞的检测性能对比

Tab.4 Detection performance comparison for 4 types of syntax rule vulnerabilities %

模型	AE			FC			AU			PU		
	Acc	P	R	Acc	P	R	Acc	P	R	Acc	P	R
VulDeePecker	66.30	70.32	68.54	65.60	71.14	69.38	64.68	69.03	70.20	70.06	75.19	70.31
SySeVR	78.74	81.12	77.68	75.69	83.11	80.03	74.14	79.27	77.77	79.43	81.21	79.45
mVulSniffer	92.60	93.02	88.30	92.74	90.69	83.17	92.10	74.77	83.17	91.50	91.07	92.69
FUNDED	93.81	94.40	92.13	93.73	95.74	94.30	94.22	90.67	92.45	92.41	91.89	92.62
VDoTR	94.04	95.14	94.05	95.17	95.33	90.15	94.35	94.79	94.64	92.05	90.48	87.83
VulHetG	95.12	96.26	95.17	96.05	96.18	93.45	94.33	94.61	94.70	93.18	92.14	92.45

结构, GGNN 更侧重于对长距离依赖关系的捕获. 针对某个漏洞模式中的错误节点, 自注意力机制会分配更高的注意力权重, 从而强化与该节点相关的依赖路径, 在不同漏洞模式下的适应能力更强.

VulHetG 对 AE 和 FC 漏洞的检测能力略高于 AU 和 PU, 证明 VulHetG 对异构特征明显的漏洞具有更强的识别能力. 原因是 AE 包含数据依赖, FC 包含控制依赖, 2 种数据存在较强的异构性, 而 AU 和 PU 存在几乎等量的依赖关系, 因此异构程度较低. 从每个数据集的横向对比结果来看, VulHetG 较门控单元和 GNN 更适合于异构特征的提取.

3.4 在 2 个真实项目数据集上的性能对比

为了验证 VulHetG 在真实项目上的泛化能力, 在真实项目数据集 REVEAL 和 Devign 上对比了方法 μ VulDeePecker、SySeVR 和 mVulSniffer. 其中, μ VulDeePecker 是粗粒度代码切片方法的代表, SySeVR 较 μ VulDeePecker 粒度更细, mVulSniffer 方法是将粗、细 2 种粒度切片结合. 不同于上述 3 种方法, VulHetG 利用注意力系数调整来代替切片操作, 实验结果如表 5、6 所示.

从表 5、6 可以看出, VulHetG 在 Devign 数据集上的检测准确率达到 80.5%, 略低于 mVulSniffer. 这是因为 mVulSniffer 在二次切片时引入函数表达式和条件分支的语法规则, 在一定程度上强化了控

制流的作用, 但对于高度依赖数据流的隐蔽型漏洞, mVulSniffer 的提升不如 VulHetG 明显, 例如 VulHetG 在 REVEAL 上的 Acc 和 F_1 较 mVulSniffer 分别高了 8% 和 11.5%.

VulHetG 的平均准确率达到 83.9%, 较 SySeVR 和 μ VulDeePecker 分别高了 40% 和 47.3%. 这是因为采用常规预处理方式难以去除真实代码场景中的冗余信息, 导致长短时记忆网络难以提取准确的漏洞语义. 相反, VulHetG 的特征构建方式产生的噪声更少, 且注意力调整机制保留了长距离依赖关系.

3.5 消融实验

为了验证 VulHetG 的特征提取方法和模型各部分的有效性, 在 CWE-119 和 CWE-399 数据集上设计 3 组消融实验, 结果如表 7 所示.

第 1 组实验采用不分层的模型策略, 利用单层 GAT 直接聚合 PDG 节点. 结果显示, 与 VulHetG 相比, 不分层的模型的准确率平均下降 6.6%. 这是因为单层 GAT 网络只在节点层传递特征, 导致依赖层特征提取不充分. 由于无须对 PDG 进行分层处理, 单个样本的检测时间较 VulHetG 大幅度减少.

第 2 组实验将 GAT 换成 GCN, Acc、 P 和 F_1 均有所下降. 原因是 GCN 通过邻接矩阵和特征矩阵的乘积聚合邻域信息, 这种类似图上滤波的操作

表 5 真实数据集 Devign 上的检测

Tab.5 Detection on real-world dataset Devign

%

模型	Acc	P	R	F_1	1-FNR	1-FPR
μ VulDeePecker	35.89	36.01	35.79	36.22	42.79	52.94
SySeVR	45.32	52.40	45.55	48.74	45.55	52.41
mVulSniffer	81.25	79.25	80.31	80.28	79.25	72.25
VulHetG	80.47	82.87	84.76	81.06	89.14	83.87

表 6 真实数据集 REVEAL 上的检测

Tab.6 Detection on real-world dataset REVEAL

%

模型	Acc	P	R	F_1	1-FNR	1-FPR
μ VulDeePecker	37.24	42.18	52.96	46.59	52.29	42.18
SySeVR	42.38	42.78	53.04	44.12	53.04	42.78
mVulSniffer	79.31	80.86	71.39	75.67	71.29	80.86
VulHetG	87.37	89.78	84.71	87.17	84.71	89.78

表 7 VulHetG 模型的消融实验

Tab.7 Ablation experiment of VulHetG model

模型	CWE-119				CWE-399			
	Acc/%	P /%	F_1 /%	DT/s	Acc/%	P /%	F_1 /%	DT/s
VulHetG (GAT)	90.99	89.06	89.47	1.330	86.41	88.83	87.79	1.397
VulHetG (GCN+self-attention)	93.33	94.02	94.03	2.054	90.89	92.31	92.00	2.010
VulHetG (非嵌入的 PDG)	91.04	93.71	93.65	2.085	90.76	90.94	89.68	2.076
VulHetG (GAT+self-attention)	96.41	96.49	96.90	2.115	94.25	94.27	94.25	2.168

在处理不同类型的图结构时,无法灵活地适应节点之间的关系变化,导致真实场景下复杂的漏洞模式无法被充分学习。

在第3组实验中,将表征形式换成基于源代码嵌入的PDG, Acc平均下降4.4%。这是因为原有的底层逻辑描述会被源代码信息干扰,其中的自定义变量、不规则的调用方式都会增加模型学习中的噪声,导致检测效果不如原模型。

4 结 语

除现有实验之外,本文在低层次漏洞上进行验证并取得了较好的检测效果,表明VulHetG具备提取细微特征进而发掘0day漏洞的潜力。VulHetG与传统的检测模型一样,都缺少0day漏洞标签、通用的数据集和通用的比较方案^[26-27],使得模型对0day漏洞的检测效果尚无定论。另外,VulHetG用于提取特征的数据模态较单一,缺乏对文档、日志、源代码等多模态数据特征的学习,对真实场景的漏洞泛化能力有待提升。

未来将尝试从真实项目中构建通用数据集,并在已有方法的基础上,采用多层注意力机制从多模态数据中提取漏洞特征,通过引入对抗样本辅助训练,获得更强的泛化能力、模型鲁棒性以及0day漏洞检测能力。

参考文献 (References):

- [1] 田笑,常继友,张弛,等. 开源软件缺陷预测方法综述[J]. 计算机研究与发展, 2023, 60(7): 1467-1488.
TIAN Xiao, CHANG Jiyu, ZHANG Chi, et al. Survey of open-source software defect prediction method [J]. *Journal of Computer Research and Development*, 2023, 60(7): 1467-1488.
- [2] ZHANG X W, ZHOU Y, TAN S H, et al. Efficient pattern-based static analysis approach via regular-expression rules [C]// *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering*. Taipa: IEEE, 2023: 132-143.
- [3] CHEN D, ZHANG Y D, WEI W, et al. Efficient vulnerability detection based on an optimized rule checking static analysis technique [J]. *Frontiers of Information Technology and Electronic Engineering*, 2017, 18(3): 332-345.
- [4] 苏小红,郑伟宁,蒋远,等. 基于学习的源代码漏洞检测研究与进展[J]. 计算机学报, 2024, 47(2): 337-374.
SU Xiaohong, ZHEN Weining, JIANG Yuan, et al. Research and progress on learning-based source code vulnerability detection [J]. *Chinese Journal of Computers*, 2024, 47(2): 337-374.
- [5] LI Z, ZOU D Q, XU S H, et al. VulDeePecker: a deep learning based system for vulnerability detection [C]// *Proceedings of the 25th Network and Distributed System Security Symposium*. San Diego: IEEE, 2018.
- [6] LI Z, ZOU D Q, XU S H, et al. SySeVR: a framework for using deep learning to detect software vulnerabilities [J]. *IEEE Transactions on Dependable and Secure Computing*, 2022, 19(4): 2244-2258.
- [7] LI Z, ZOU D Q, XU S H, et al. VulDeeLocator: a deep learning-based fine-grained vulnerability detector [J]. *IEEE Transactions on Dependable and Secure Computing*, 2022, 19(4): 2821-2837.
- [8] WU Y M, ZOU D Q, DOU S H, et al. VulCNN: an image inspired scalable vulnerability detection system [C]// *Proceedings of the 44th International Conference on Software Engineering*. Pittsburgh: ACM, 2022: 2365-2376.
- [9] ZHOU Y Q, LIU S Q, DU X N, et al. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks [J]. *Neural Information Processing Systems*, 2019, 2019(32): 10197-10207.
- [10] WANG H T, YE G X, TANG Z Y, et al. Combining graph-based learning with automated data collection for code vulnerability detection [J]. *IEEE Transactions on Information Forensics and Security*, 2021, 2021(16): 1943-1958.
- [11] FAN Y H, WAN C H, FU C, et al. VDoTR: vulnerability detection based on tensor representation of comprehensive code graphs [J]. *Computers and Security*, 2023, 2023(130): 103247.
- [12] ALOMAR E A, ALOMAR S A, MKAOUER M W. On the use of static analysis to engage students with software quality improvement: an experience with PMD [C]// *Proceeding of 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training*. Melbourne: IEEE, 2023: 179-191.
- [13] BARTA B, MANZ G, SIKET I, et al. Challenges of sonarqube plug-in maintenance [C]// *Proceedings of the 26th International Conference on Software Analysis, Evolution, and Reengineering*. Hangzhou: IEEE, 2019: 574-578.
- [14] PERL H, DECHAND S, SMITH M, et al. VCCFinder: finding potential vulnerabilities in open-source projects to assist code audits [C]// *Proceedings of the 22nd ACM Conference on Computer and Communications Security*. Denver: ACM, 2015: 426-437.
- [15] ZOU D Q, XU S H, WANG S J, et al. μ VulDeePecker: a deep learning-based system for multiclass vulnerability detection [J]. *IEEE Transactions on Dependable and Secure Computing*, 2019, 18(5): 2224-2236.
- [16] FENG Q, FENG C D, HONG W J, et al. Graph neural network-based vulnerability predication [C]// *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution*. Adelaide: IEEE, 2020: 800-801.
- [17] 徐泽鑫,段立娟,王文健,等. 基于上下文特征融合的代码漏洞检测方法[J]. 浙江大学学报: 工学版, 2022, 56(11): 2260-2270.
XU Zexin, DUAN Lijuan, WANG Wenjian, et al. Code vulnerability detection method based on contextual feature fusion [J]. *Journal of Zhejiang University: Engineering Science*,

